

PID Tuner

Tuning & Identification Methods Reference

A precise description of every method implemented in the app,
matched to the formulas in `tune.py` and `identify.py`.

This document specifies the nine controller-tuning methods and the three plant-identification primitives that feed them. Each entry states the governing equations exactly as coded, the assumptions behind them, when the method applies, and when it refuses or breaks down. Sources are the course handouts (HO13 AMIGO, HO14 SIMC, problem session PS4) and the primary literature cited per method.

Contents

1	Conventions	1
2	Identification primitives	2
2.1	FOPDT step fit (28.3 % / 63.2 % method)	2
2.2	Relay-feedback test (Aström–Hägglund, 1984)	3
2.3	Bode-based ultimate gain (analytical)	3
3	Tuning methods	3
3.1	1. Stable pole cancellation	3
3.2	2. Ziegler–Nichols Method I (reaction curve)	4
3.3	3. Ziegler–Nichols Method II (ultimate gain)	4
3.4	4. AMIGO (Aström–Hägglund, 2006)	4
3.5	5. SIMC (Skogestad, “Simple IMC”)	5
3.6	6. Boyd convex–concave design	5
3.7	7. Cohen–Coon (1953)	6
3.8	8. Chien–Hrones–Reswick (CHR, 1952)	6
3.9	9. Tyreus–Luyben (Luyben & Luyben, 1997)	7
4	Post-processing: halve gains	7
5	Benchmark behavior	8
6	Comparing methods: objective criteria	8
	References	9

1 Conventions

The controller is stored internally in **parallel form**

$$D(s) = K_P + \frac{K_I}{s} + K_D s,$$

while most tuning rules are published in **textbook (standard) form**

$$D(s) = K_P \left(1 + \frac{1}{T_I s} + T_D s \right).$$

The two are related by

$$K_I = \frac{K_P}{T_I}, \quad K_D = K_P T_D, \quad \Longleftrightarrow \quad T_I = \frac{K_P}{K_I}, \quad T_D = \frac{K_D}{K_P}.$$

A rule that yields (K_P, T_I, T_D) is converted to (K_P, K_I, K_D) on return; both forms are shown in the result panel. Plant polynomials are stored in descending powers of s (scipy / MATLAB convention), and dead time L enters as a factor e^{-Ls} .

The plant model throughout the identification step is **first-order plus dead time** (FOPDT):

$$G(s) = \frac{A e^{-Ls}}{\tau s + 1},$$

with static gain A (also written K), time constant τ , and dead time L .

2 Identification primitives

Four of the six tuning methods (ZN-I, ZN-II, AMIGO, SIMC) operate on a reduced model rather than the raw transfer function. These primitives produce that model.

2.1 FOPDT step fit (28.3 % / 63.2 % method)

The plant is simulated against an open-loop step of amplitude Δu , and A, τ, L are read off the response. For a true FOPDT system the response is

$$y(t) = A \Delta u \left[1 - e^{-(t-L)/\tau} \right], \quad t \geq L,$$

which reaches 28.3% of its final change at $t = L + \tau/3$ and 63.2% at $t = L + \tau$. Solving for the two timestamps $t_{28.3}$ and $t_{63.2}$ gives

$$\tau = 1.5 (t_{63.2} - t_{28.3}), \quad L = t_{63.2} - \tau, \quad A = \frac{\Delta y_\infty}{\Delta u}.$$

Using two crossing times (Sundaresan–Krishnaswamy) is more noise-robust than the classical Ziegler–Nichols tangent construction, whose slope depends on a numerical derivative at the inflection point.

Applies when: Any plant that settles to a finite steady state under a step.

Refuses / fails when: The step produces no discernible change ($\Delta y_\infty \approx 0$); a placeholder model is returned and downstream rules will report a degenerate fit.

2.2 Relay-feedback test (Aström–Hägglund, 1984)

A hysteretic on/off relay of amplitude h drives the plant into a sustained limit cycle. The describing-function approximation of the relay gives the ultimate gain and period directly from the cycle:

$$K_u = \frac{4h}{\pi a}, \quad P_u = \text{period of the steady-state limit cycle},$$

where a is half the peak-to-peak amplitude of the output oscillation. The implementation discretizes the plant, applies the relay with optional hysteresis, and measures P_u from the mean of the last several half-periods (zero-crossings) and a from the tail extrema.

Applies when: The plant sustains a limit cycle under relay feedback (most processes with $\geq 180^\circ$ of attainable phase lag).

Refuses / fails when: Fewer than four zero-crossings are detected, or the oscillation amplitude is zero — the routine raises and suggests the Bode-based method instead.

2.3 Bode-based ultimate gain (analytical)

When the closed-form model is available (it always is here, since the user supplied it), K_u and P_u are found by locating the -180° phase crossover directly rather than by simulating a relay:

$$\angle G(j\omega_{180}) = -180^\circ \implies K_u = \frac{1}{|G(j\omega_{180})|}, \quad P_u = \frac{2\pi}{\omega_{180}}.$$

The smallest positive crossover is bracketed on a logarithmic frequency grid and refined by bisection on a continuous (anchored) phase function to avoid the $(-\pi, \pi]$ wraparound. This is faster and more accurate than the relay test.

Applies when: The plant phase actually crosses -180° (e.g. third-order or higher, or any order with dead time).

Refuses / fails when: No finite crossover exists (e.g. a pure first- or second-order lag without dead time) — then K_u is infinite and ZN-II is not applicable; the routine raises and points to ZN-I.

3 Tuning methods

3.1 1. Stable pole cancellation

The course quick-and-dirty method (PS4). The PID controller carries two zeros, which are placed exactly on two stable plant poles at $s = -p_1$, $s = -p_2$:

$$C(s) = K_D \frac{(s + p_1)(s + p_2)}{s}.$$

Expanding the numerator, $C(s) = K_D s + (p_1 + p_2)K_D + p_1 p_2 K_D / s$, so the parallel gains read off as

$$K_D \text{ (free)}, \quad K_P = (p_1 + p_2) K_D, \quad K_I = p_1 p_2 K_D.$$

After cancellation the open loop collapses to $K_D N(s) / [s R(s)]$, an integrator times the leftover plant dynamics. K_D is the single free knob and sets the closed-loop speed. By default the app cancels the two *slowest* stable poles (smallest $|\text{Re}|$), which yields the fastest clean response; a complex-conjugate pair is cancelled together so the gains stay real.

Applies when: The plant has at least two stable poles, and those poles are known / fixed (not drifting with operating point).

Refuses / fails when: Either chosen pole is in the right half-plane or on the imaginary axis. RHP cancellation is rejected outright — imperfect cancellation leaves a hidden unstable mode, the cardinal sin emphasized in lecture (“never, ever do RHP cancellations”).

3.2 2. Ziegler–Nichols Method I (reaction curve)

Classical 1942 open-loop rules driven by the FOPDT fit. With reaction rate $R = A/\tau$ and dead time L , the PID row of the ZN-I table is

$$K_P = \frac{1.2}{R L} = \frac{1.2 \tau}{A L}, \quad T_I = 2L, \quad T_D = 0.5L.$$

The rules were derived for disturbance rejection and characteristically overshoot ($\sim 40\%$ on the benchmark) when used for setpoint tracking — hence the *Halve gains* toggle (§4).

Inputs: A, τ, L from the FOPDT step fit.

Applies when: Self-regulating plants with a clear S-shaped step response.

Refuses / fails when: Degenerate FOPDT fit ($A = 0$ or $\tau = 0$).

3.3 3. Ziegler–Nichols Method II (ultimate gain)

Closed-loop rules driven by K_u, P_u (from the relay test or the Bode crossover). The PID row is

$$K_P = 0.6 K_u, \quad T_I = 0.5 P_u, \quad T_D = 0.125 P_u.$$

Like ZN-I, this is a disturbance-rejection design and overshoots heavily on tracking (the most aggressive of the six on the benchmark); the *Halve gains* toggle tempers it.

Inputs: K_u, P_u from §2.2 or §2.3.

Applies when: Plants that admit a finite ultimate gain (phase crosses -180°).

Refuses / fails when: No finite K_u (see §2.3).

3.4 4. AMIGO (Aström–Hägglund, 2006)

Robust rules from handout HO13, tuned to give roughly 20% overshoot under a constraint on the peak sensitivity — more conservative than ZN. For the non-integrating FOPDT $G(s) = Ae^{-Ls}/(\tau s + 1)$:

$$K_P = \frac{1}{A} \left(0.2 + 0.45 \frac{\tau}{L} \right), \quad T_I = \frac{0.4L + 0.8\tau}{L + 0.1\tau} L, \quad T_D = \frac{0.5\tau}{0.3L + \tau} L.$$

For an integrating process $G(s) = Ae^{-Ls}/s$ the rules switch to

$$K_P = \frac{0.45}{A}, \quad T_I = 8L, \quad T_D = 0.5L.$$

Inputs: A, τ, L from the FOPDT fit (plus an integrating/self-regulating flag).

Applies when: FOPDT or integrating-plus-dead-time plants where a robust, moderate-overshoot response is wanted.

Refuses / fails when: Zero static gain ($A = 0$).

3.5 5. SIMC (Skogestad, “Simple IMC”)

Internal-model-control rules from handout HO14 with a single tuning knob, the desired closed-loop time constant τ_c (default $\tau_c = L$, the “fast and robust” choice). For FOPDT this gives a **PI** controller:

$$K_P = \frac{1}{A} \cdot \frac{\tau}{\tau_c + L}, \quad T_I = \min(\tau, 4(\tau_c + L)), \quad T_D = 0.$$

The min in T_I is Skogestad’s anti-sluggishness fix: it caps the integral time so disturbance rejection on lag-dominant plants does not become unacceptably slow. Supplying a second time constant τ_2 from a second-order (SOPDT) fit promotes it to a **PID** controller:

$$K_P = \frac{1}{A} \cdot \frac{\tau_1}{\tau_c + L}, \quad T_I = \min(\tau_1, 4(\tau_c + L)), \quad T_D = \tau_2.$$

Inputs: A, τ, L from the FOPDT fit; optional τ_2 and τ_c .

Applies when: Lag-dominant processes; the default produces the smoothest, lowest-overshoot tracking of the six methods on the benchmark.

Refuses / fails when: Zero static gain ($A = 0$).

3.6 6. Boyd convex–concave design

From Hast, Aström, Bernhardsson & Boyd, *PID Design by Convex–Concave Optimization* (ECC 2013). Unlike the others, this operates on the **true plant frequency response**, so it handles arbitrary-order plants without an FOPDT proxy. The loop transfer $L(j\omega) = (K_P + K_I/(j\omega) + K_D j\omega) P(j\omega)$ is *affine* in (K_P, K_I, K_D) at each frequency. The design maximizes the integral gain subject to robustness constraints:

$$\max_{K_P, K_I, K_D} K_I \quad \text{s.t.} \quad \|S\|_\infty \leq M_s, \quad \|T\|_\infty \leq M_t,$$

where the sensitivity peak M_s and complementary-sensitivity peak M_t (both default 1.4) become forbidden circles in the Nyquist plane:

$$|L(j\omega) - c| \geq r, \quad \begin{cases} c = -1, & r = 1/M_s \quad (\text{sensitivity}) \\ c = -\frac{M_t^2}{M_t^2 - 1}, & r = \frac{M_t}{M_t^2 - 1} \quad (\text{comp. sens.}) \end{cases}$$

The constraint $|L - c| \geq r$ is concave; linearizing it about the current iterate turns each step into a linear program. Iterating to convergence yields the gains. The solver is seeded from SIMC (a good seed matters, since the LP bounds are scaled to it).

Inputs: The full plant transfer function, M_s , M_t , and an optional derivative on/off flag.

Applies when: Any stable plant; especially valuable for high-order or resonant plants where an FOPDT reduction throws away too much.

Refuses / fails when: The LP becomes infeasible (constraints too tight for the plant) — the iteration stops at the last feasible iterate.

3.7 7. Cohen–Coon (1953)

Open-loop reaction-curve rules, like ZN-I but using the time constant τ and dead time L separately (ZN-I collapses both into the single slope $R = A/\tau$). A built-in dead-time correction makes them noticeably better than ZN-I on delay-dominant plants (large L/τ). With $r = L/\tau$ the PID row is

$$K_P = \frac{1}{A} \frac{\tau}{L} \left(\frac{4}{3} + \frac{r}{4} \right), \quad T_I = L \frac{32 + 6r}{13 + 8r}, \quad T_D = \frac{4L}{11 + 2r}.$$

Like ZN, the design targets a quarter-amplitude-decay load response, so it is aggressive on setpoint steps (largest overshoot of the FOPDT methods on the benchmark).

Inputs: A, τ, L from the FOPDT step fit.

Applies when: Self-regulating plants, particularly when dead time is a sizeable fraction of the time constant.

Refuses / fails when: Degenerate FOPDT fit ($A = 0$ or $\tau = 0$).

3.8 8. Chien–Hrones–Reswick (CHR, 1952)

A refinement of ZN-I that lets the designer choose two things the older rules fix implicitly: whether the loop is tuned for *setpoint tracking* (servo) or *load rejection* (regulator), and whether to target the quickest *aperiodic* response (0% overshoot) or the quickest response with 20% overshoot. The four PID rows, all driven by the FOPDT fit:

variant	K_P	T_I	T_D
setpoint, 0%	$0.60 \frac{\tau}{AL}$	τ	$0.50 L$
setpoint, 20%	$0.95 \frac{\tau}{AL}$	1.40τ	$0.47 L$
load, 0%	$0.95 \frac{\tau}{AL}$	$2.40 L$	$0.42 L$
load, 20%	$1.20 \frac{\tau}{AL}$	$2.00 L$	$0.42 L$

A caveat worth stating in teaching: the “0% / 20% overshoot” labels describe the response of the *nominal FOPDT model* from which the rules were derived. Applied to a true higher-order plant, the realized overshoot can differ from the label (sometimes the 20% row is actually *less* oscillatory than the 0% row, because the FOPDT reduction distorts the apparent dead time). This mismatch is itself a useful demonstration of how much the model-reduction step matters.

Inputs: A, τ, L from the FOPDT fit, plus the servo/regulator and 0%/20% selections.

Applies when: Self-regulating plants where the designer has a clear preference for tracking vs. rejection, or for damping vs. speed.

Refuses / fails when: Degenerate FOPDT fit, or an unknown variant request.

3.9 9. Tyreus–Luyben (Luyben & Luyben, 1997)

The ultimate-gain rule popularized in Luyben & Luyben, *Essentials of Process Control* (McGraw-Hill, 1997). Like ZN-II it works from K_u and P_u , but it is deliberately detuned — a smaller proportional gain and a much longer integral time — for larger stability margins and far less oscillation, at the cost of speed. This suits the long-dead-time and near-integrating loops common in process control. The PID and PI rows:

$$\text{PID: } K_P = \frac{K_u}{2.2}, \quad T_I = 2.2 P_u, \quad T_D = \frac{P_u}{6.3}; \quad \text{PI: } K_P = \frac{K_u}{3.2}, \quad T_I = 2.2 P_u.$$

Because the rule is already conservative, the *Halve gains* toggle is neither needed nor recommended here.

Inputs: K_u, P_u from the Bode crossover or relay test (§2.2–2.3); PID/PI selector.

Applies when: Plants with a finite ultimate gain, especially dead-time-dominant or integrating loops where ZN-II would be far too oscillatory.

Refuses / fails when: No finite K_u (see §2.3).

4 Post-processing: halve gains

The ZN rules target disturbance rejection and overshoot on setpoint steps. The Emami / Åström recommendation is to soften them; the app exposes a toggle that halves all three gains:

$$(K_P, K_I, K_D) \longmapsto \frac{1}{2}(K_P, K_I, K_D).$$

Strictly, the classical recommendation halves K_P alone; halving all three is a slightly different (and simpler) softening. Since the gains are a free knob anyway, the override is offered for any method, not just ZN.

5 Benchmark behavior

All methods are exercised in the test suite against $G(s) = 1000/[(s + 1)(10s + 1)] \cdot e^{-0.5s}$. Representative step-tracking results:

Method	Overshoot	t_s (2%)	IAE
Pole cancellation	$\approx 0\%$	34	9.95
Ziegler–Nichols I	41%	14.7	3.68
Ziegler–Nichols II	78%	10.7	3.51
AMIGO	19%	20.2	4.96
SIMC	4%	11.3	3.44
Boyd ($M_s = M_t = 1.4$)	18%	19.2	4.89
Cohen–Coon	44%	8.2	3.22
CHR setpoint, 0%	5%	20.7	3.82
CHR setpoint, 20%	2%	4.8	2.56
CHR load, 0%	35%	15.2	3.84
CHR load, 20%	45%	13.6	3.68
Tyreus–Luyben	13%	16.0	2.51

The pattern matches each method’s intent: pole cancellation, SIMC, and the CHR setpoint rows are smooth; the ZN and Cohen–Coon rows and the CHR *load* rows are fast but oscillatory (the load rows are tuned for disturbance rejection, not tracking, so they overshoot a setpoint step); AMIGO and Boyd land near their $\sim 20\%$ robustness targets; and Tyreus–Luyben sits between ZN-II and the robust methods, as its conservative design intends.

6 Comparing methods: objective criteria

No single number declares one method “best” — tuning is multi-objective, and the ranking depends on the plant *and* on what is being asked of the loop. The app’s **Compare all methods** view computes the following metrics for every method and presents them as a colour-coded heatmap (green = best, red = worst, per column) and a radar chart (each axis normalized so the outer edge is best). Every metric below is oriented so that *lower is better*.

Tracking performance (unit setpoint step)

- **Overshoot** and **2% settling time** — the classic step-response shape numbers.
- **IAE** $= \int_0^\infty |e(t)| dt$, the integral of absolute error. Two relatives are worth knowing: **ISE** $= \int e^2 dt$ punishes large early errors and tends to produce fast, oscillatory tunings; **ITAE** $= \int t |e| dt$ punishes errors that *persist*, rewarding well-damped responses and usually tracking subjective “looks good” best.

Disturbance rejection (unit load step at the plant input)

- **IAE_{load}** $= \int |y(t)| dt$ with the setpoint held at zero and a step injected at the plant input. Most classical rules (ZN, Cohen–Coon, the CHR *load* rows) were designed for this objective,

so judging them only on setpoint tracking is unfair; the two columns routinely disagree about the winner.

Robustness (loop frequency response $L = C P$)

- **Maximum sensitivity** $M_s = \max_{\omega} |S(j\omega)| = \max_{\omega} |1/(1 + L)|$. This is the single most-recommended robustness scalar: it is the inverse of the shortest distance from the Nyquist curve to -1 , bundling gain and phase margin into one number. The usual “robust enough” band is $M_s \in [1.4, 2.0]$ ($1.4 \approx$ conservative, $2.0 \approx$ aggressive).
- **Complementary sensitivity peak** $M_t = \max_{\omega} |T(j\omega)|$, and the classical **gain** and **phase margins**, reported alongside.

Actuator effort / smoothness

- **Total variation** $TV(u) = \sum_k |u_k - u_{k-1}|$, a measure of how much the control signal moves — high values mean chattering or noise amplification and real wear on a physical actuator. The derivative-heavy methods (ZN-II, Cohen–Coon, Tyreus–Luyben) tend to score worst here. Peak $|u|$ is also reported.

Making “best” well-posed

The principled way to compare is *constrained*: minimise a performance index subject to a robustness bound — e.g. minimise IAE_{load} subject to $M_s \leq 1.6$. This is exactly what AMIGO and Boyd do internally, and it converts the ill-posed “which is best?” into the well-posed “which is best at a fixed robustness level?” The honest answer is then not a single winner but a **Pareto frontier**: plotting each method in the (M_s, IAE_{load}) plane, the non-dominated set traces the achievable trade-off, and any method up-and-to-the-right of another is strictly dominated. The radar chart shows all axes at once (bigger polygon = better all round); the heatmap shows the raw numbers; the Pareto view isolates one head-to-head trade-off. The plant decides the ranking: dead-time-dominant plants favour SIMC and Tyreus–Luyben, clean lag-dominant plants favour pole-cancellation and the CHR setpoint rows, and resonant or high-order plants favour Boyd.

1. J. G. Ziegler and N. B. Nichols, “Optimum settings for automatic controllers,” *Trans. ASME*, 1942.
2. K. J. Aström and T. Hägglund, “Automatic tuning of simple regulators with specifications on phase and amplitude margins,” *Automatica*, 1984 (relay method).
3. K. J. Aström and T. Hägglund, *Advanced PID Control*, ISA, 2006 (AMIGO) — course handout HO13.
4. S. Skogestad, “Simple analytic rules for model reduction and PID controller tuning,” *J. Process Control*, 2003 (SIMC) — course handout HO14.
5. M. Hast, K. J. Aström, B. Bernhardsson and S. Boyd, “PID design by convex–concave optimization,” *European Control Conference*, 2013.
6. G. H. Cohen and G. A. Coon, “Theoretical consideration of retarded control,” *Trans. ASME* 75, 827–834, 1953.
7. K. L. Chien, J. A. Hrones and J. B. Reswick, “On the automatic control of generalized passive systems,” *Trans. ASME* 74, 175–185, 1952.

-
8. W. L. Luyben and M. L. Luyben, *Essentials of Process Control*, McGraw-Hill, 1997 (Tyreus–Luyben rules; orig. B. D. Tyreus and W. L. Luyben, *Ind. Eng. Chem. Res.* 31, 2625–2628, 1992).
 9. Course problem session PS4 — stable pole cancellation.